Lecture 3: Representation Learning (Part 1)

Complete Lecture Notes & Mathematical Derivations

Machine Learning Theory — Professor Arun Rajkumar

---

1. Core Philosophy & Rationale

- **Goal of Unsupervised Learning:** Given an unlabelled dataset of n feature vectors $x_1, x_2, \ldots, x_n \in \mathbb{R}^d$ (d features), the objective is to discover meaningful underlying structures without target labels.
- **"Comprehension is Compression":** Grounded in computer scientist George Chaitin's principle, *learning or comprehending data is equivalent to compressing it.* To understand a dataset means distilling raw features into a compact representation while retaining essential information.
- **The Diagnostic Question:** For every machine learning algorithm encountered in the course, always ask:

"Where is the compression really happening in this algorithm?"

2. Step 1: Exact Reconstruction ($y = 2x$)

- **The Blackboard Problem:** Consider $n = 4$ data points in $d = 2$ dimensions:

$$x_1 = \begin{bmatrix} -7 \\ -14 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 2.5 \\ 5 \end{bmatrix}, \quad x_3 = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix}, \quad x_4 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

- **Naive Storage:** Storing all 4 vectors independently requires $n \times d = 4 \times 2 = \mathbf{8}$ real numbers.
- **Exploiting Feature Relationships ($x_2 = 2x_1$):** Decomposing the dataset into:
  1. **1 Shared Representative Vector ($w \in \mathbb{R}^2$):** $w = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ (2 real numbers).
  2. **1 Scalar Coefficient per Point ($c_i \in \mathbb{R}$):** $c_1 = -7,\ c_2 = 2.5,\ c_3 = 0.5,\ c_4 = 0$ (4 real numbers).
- **Compressed Storage:** Total required storage drops to $d + n = 2 + 4 = \mathbf{6}$ real numbers.
- **Scaling Analysis ($n \to \infty$):** For $n = 1$ billion points in 2D space, naive storage requires $2n = 2$ billion numbers, whereas compressed storage requires $n + d = 1$ billion $+ 2$ numbers—achieving **nearly a 50% compression rate**.
- **Exact Reconstruction Formula:** $x_i = c_i \cdot w = \begin{bmatrix} c_i w_1 \\ c_i w_2 \end{bmatrix}$.

3. Step 2: Off-Line Data & The Multi-Basis Fallacy

- **The Off-Line Point:** Consider points lying on line $y = x$, with an off-line point $x_5 = \begin{bmatrix} 1.5 \\ 3.0 \end{bmatrix}$.
- **Why Adding Basis Vectors Fails:** Attempting exact reconstruction by adding a second basis vector requires storing 2 scalar coefficients per point plus 4 basis coordinates. Total storage balloons to $2n + 4$ numbers, which is **worse than uncompressed naive storage ($2n$)**.
- **The Compression Trade-off:** To maintain compression on real-world noisy data, **exact reconstruction must be sacrificed** in favor of mapping off-line points to a **proxy point** on the line.

4. Step 3: Derivation of Optimal Proxy via Orthogonal Projection

Let an off-line data point be $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ and the line direction be $w = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$. Any proxy point on the line has the form $c \cdot w = \begin{bmatrix} cw_1 \\ cw_2 \end{bmatrix}$.

1. **Error Distance Objective Function:**

$$E(c) = \|x - cw\|^2 = (x_1 - cw_1)^2 + (x_2 - cw_2)^2$$

2. **Solving for Optimal Scalar c^*:** Taking the derivative with respect to c and setting it to zero:

$$\frac{dE}{dc} = -2w_1(x_1 - cw_1) - 2w_2(x_2 - cw_2) = 0 \implies c(w_1^2 + w_2^2) = x_1w_1 + x_2w_2$$

$$\mathbf{c^* = \frac{x_1w_1 + x_2w_2}{w_1^2 + w_2^2} = \frac{x^Tw}{\|w\|^2}}$$

3. **Unit Vector Simplification ($\|w\| = 1 \implies w_1^2 + w_2^2 = 1$):**

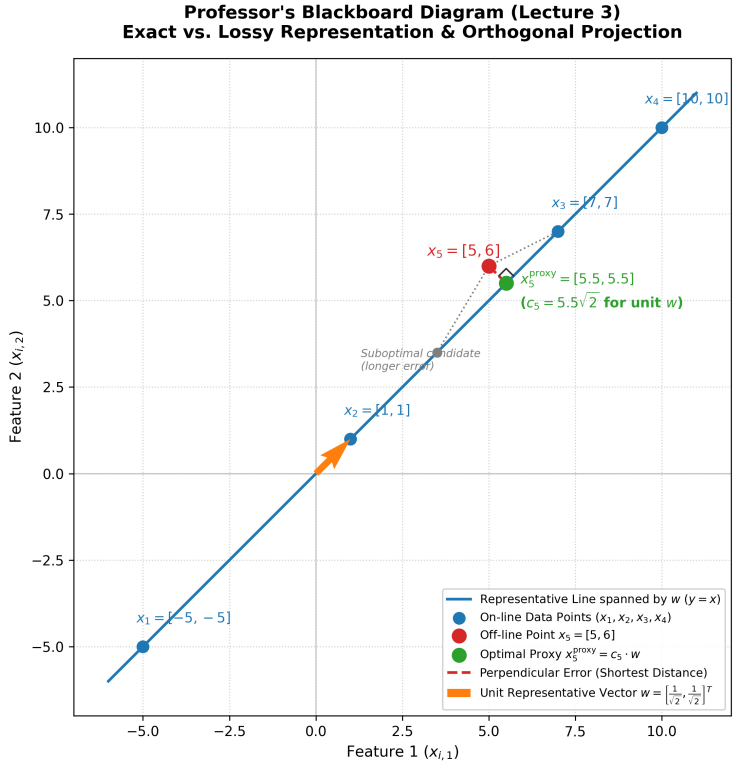
$$c^* = x_1w_1 + x_2w_2 = x^Tw \quad \text{(1D Compressed Scalar Coordinate)}$$

4. **The Final Answer — Reconstructed Proxy Vector:** Re-multiplying scalar code c^* by direction vector w recovers the final proxy point back in 2D feature space:

$$\mathbf{x_{proxy} = c^* \cdot w} = \begin{bmatrix} c^*w_1 \\ c^*w_2 \end{bmatrix} = \begin{bmatrix} \mathbf{x_1w_1 + x_2w_2} \\ \mathbf{w_1^2 + w_2^2} \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$$

5. Professor's Blackboard Diagram

The diagram below reconstructs Professor Arun Rajkumar's blackboard drawing, illustrating on-line points $x_1 \dots x_4$, off-line point $x_5 = \begin{bmatrix} 1.5 \\ 3.0 \end{bmatrix}$, and its optimal proxy point $x_5^{\text{proxy}} = \begin{bmatrix} 5.5 \\ 5.5 \end{bmatrix}$ obtained via orthogonal projection (90° right angle).



SUMMARY & PROJECTION FORMULAS (PROFESSOR'S NOTATION)

- **Projection of x_5 on w:**

$$\text{Projection of } x_5 \text{ on } w = c \cdot w$$

- **Where c is the scalar coordinate:**

$$c = \frac{x^T w}{\|w\|^2} = \frac{x_1 w_1 + x_2 w_2}{w_1^2 + w_2^2}$$

- **Therefore, Projection of x on w (General Formula):**

$$\text{Projection of } x \text{ on } w = \left[\frac{x^T w}{\|w\|^2}\right] \cdot w = \left[\frac{x_1 w_1 + x_2 w_2}{w_1^2 + w_2^2}\right] \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$$

- **When w is a Unit Vector ($\|w\| = 1 \implies w_1^2 + w_2^2 = 1$):**

$$\text{1D Compressed Scalar Code } (c^*): \quad c^* = x_1 w_1 + x_2 w_2 = x^T w$$

$$\text{Final Proxy Vector } (x_{\text{proxy}}): \quad x_{\text{proxy}} = (x^T w) \cdot w = \begin{bmatrix} (x_1 w_1 + x_2 w_2) w_1 \\ (x_1 w_1 + x_2 w_2) w_2 \end{bmatrix}$$